

Лабораторная работа: «Реализация драйверов для устройств»

Цель работы

Введение

Драйвер устройства содержит набор программных подпрограмм, которые позволяют компьютерным программам получать доступ к периферийному устройству без необходимости для программиста быть знакомым с низкоуровневыми деталями функционирования периферийного устройства. Простым примером периферийного устройства, которое мы используем в компьютерах, является клавиатура, которая может быть подключена к компьютеру через интерфейс PS/2 или USB. Программистам, которые пишут программу на языке Си для считывания строки символов с клавиатуры персонального компьютера, не обязательно знать, как их программа воспринимает нажатия клавиш на клавиатуре. Например, для считывания целого числа с клавиатуры программист на языке Си использует инструкцию:

```
scanf("%d",&some_variable);
```

Целью драйверов устройств является обеспечение высокого уровня абстракции для периферийных устройств. Реализация драйверов устройств часто требует от программиста больших знаний не только о том, как работает периферийное устройство, но и о контексте, в котором оно используется.

Компьютерная мышь – это еще один пример простого устройства, которое можно подключить через интерфейс PS/2 или USB. Сегодня мышь, скорее всего, будет подключаться через USB-соединение. Ранее было распространено использование интерфейса PS/2. Поскольку интерфейс PS/2 намного проще, мы будем использовать его в этом лабораторном упражнении.

Практическая часть

Часть 1. Опрос данных мыши через PS/2

В этой части вы сделаете первые шаги по взаимодействию компьютерной мышью. Для этого вы должны иметь возможность отправлять и получать информацию с мыши через порт PS/2 на компьютере серии DE. Порт PS/2 в компьютере реализован с помощью IP-ядра с портом PS/2. Это ядро включает в себя 256-байтовый буфер ввода-вывода (FIFO), в котором хранятся данные, полученные от устройства PS/2. Интерфейс к порту PS/2 в компьютере имеет два регистра, отображаемых в памяти, как показано на рисунке №1.

Address	31	...	16	15	...	10	9	8	7	...	1	0	
0xFF200100	RAVAIL			RVALID	Unused				Data				PS2_Data
0xFF200104							CE		RI			RE	PS2_Control

Рисунок №1. Формат регистров управления портом PS/2.

Регистр данных PS2 состоит из полей RAVAIL, RVALID и Data. Если RVALID = 1, поле RAVAIL указывает количество байт данных, полученных через порт PS/2, которые в данный момент хранятся в буфере FIFO. Первый байт буфера может быть считан через поле данных. Обратите внимание, что чтение любой части регистра данных PS2 приводит к извлечению первого байта из буфера FIFO. Управляющий регистр PS2 используется для включения прерываний для порта PS/2. Прерывания можно включить, установив значение в поле RE. Когда RE = 1, порт PS/2 генерирует запрос на прерывание, когда значение

RAVAIL превышает 0. Во время ожидания прерывания для поля RI устанавливается значение 1, и его можно очистить, очистив FIFO-файл PS/2. Поле CE устанавливается портом PS/2, если при отправке команды на устройство PS/2 возникает ошибка. Мы будем использовать этот регистр в последующих частях упражнения. При первой попытке установить связь с помощью мыши вы будете использовать опрос для получения данных с порта PS/2. Выполните следующие действия:

1. Напишите программу на языке Си, которая опрашивает регистр данных PS/2 и отображает последние три байта, полученные на 7-сегментные дисплеи.
 - a) Ваша программа должна сначала включить связь с мышью, отправив байт 0xF4 в регистр данных PS/2.
 - b) Затем ваша программа должна непрерывно считывать содержимое регистра данных PS/2.
 - c) Каждый раз проверяйте RVALID поле, чтобы определить, является ли байт в поле данных допустимыми данными.
 - d) Только если RVALID поле равно 1, вы должны сохранять данные и отображать их на 7-сегментных дисплеях.
2. Создайте новый проект в Intel FPGA Monitor Program для компьютера Intel серии DE и добавьте свой исходный код.
3. Скомпилируйте и загрузите свою программу на плату серии DE.
4. Подключите мышь PS/2 к порту PS/2 на плате.
5. Запустите программу и следите за сообщениями, получаемыми от устройства компьютерной мыши, при её перемещении и нажатии кнопок.

Часть 2. Инициализация и сброс мыши

В этой части вы улучшите свою программу, чтобы правильно инициализировать устройство мыши. Чтобы сделать это, вам нужно будет понять некоторые команды, которые могут быть отправлены мышью (например, команду 0xF4, которую мы использовали в части 1), а также сообщения, которые мышь может отправлять в ответ на эти команды. Необходимые команды и реакция мыши на них приведены в таблице №1.

Command Description	Command Byte	Response
Reset the mouse to default mode	0xFF	Responds with a 0xFA message, followed by a 2-byte message 0xAA00 if successful. A byte 0xFC will be sent otherwise to indicate an error.
Enable Mouse to send position and button status messages	0xF4	Responds with a single 0xFA byte if successful.
Disable Mouse	0xF5	Responds with a single 0xFA byte if successful. Send the Enable mouse command to resume receiving messages about the user's interactions with the mouse.

Таблица №1. Некоторые команды компьютерной мыши PS/2 с описанием ожидаемого ответа.

Чтобы обеспечить правильную работу устройства mouse, важно перезагрузить мышь и включить ее, прежде чем пытаться обрабатывать какие-либо сообщения. Таким образом, ваша программа должна сначала отправить команду сброса мыши и дождаться ответа. Если сброс прошел успешно, мышь отправит 3-байтовую последовательность, равную 0xFAAA00. После получения этого ответа от мыши отправьте команду Включить мышь и дождитесь подтверждения 0xFA.

Перезагрузка мыши может завершиться неудачей из-за неисправности, ошибки связи или из-за того, что мышь не подключена к системе. Если мышь работает со сбоями и не реагирует на команды, это может рассматриваться как отсутствие подключения. Если она ответит байтом 0xFC, повторно отправьте команду сброса, чтобы убедиться, что мышь правильно сброшена, прежде чем продолжить.

Отключенная мышь может быть подключена в любой момент и должна быть должным образом инициализирована. К счастью, когда мышь подключена к порту PS/2, она автоматически перезагружается и отправляет 2-байтовое сообщение 0xAA00, как если бы на нее была отправлена команда сброса. Ваша программа должна следить за появлением сообщения 0xAA00 от мыши. Если эта последовательность обнаружена, ваша программа должна отправить мыши команду сброса, за которой следует команда включения мыши.

Напишите программу на языке Си, которая взаимодействует с мышью через порт PS/2 на компьютере серии DE. Ваша программа должна корректно перезагружать и включать устройство мыши, а также обрабатывать случай повторного подключения мыши к системе во время работы программы.

Часть 3. Обработка пакетов перемещения мыши

В этой части вы узнаете, как обрабатывать сообщения, отправляемые устройством мыши, чтобы указать изменение его положения и состояния кнопок. Изменение положения обозначается двумя 9-разрядными цифрами со знаком, одна из которых обозначает изменение горизонтального положения, а другая - вертикального. Перемещение мыши вправо приводит к положительному изменению положения мыши по горизонтали, а перемещение мыши влево обозначается отрицательным значением. Аналогично, когда вы перемещаете мышь вперед, изменение вертикального положения является положительным, а когда вы тянете ее на себя, оно является отрицательным. Состояние кнопок мыши определяется с помощью трех разрядов, по одному на кнопку в обычной трехкнопочной мыши. При нажатии кнопки соответствующий бит устанавливается равным 1, в противном случае бит устанавливается равным 0.

По умолчанию мышь, которая включена, отправляет 3-байтовые пакеты через порт PS/2, чтобы сообщить о своем состоянии. Формат пакета приведен в таблице №2.

	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
byte 1	<i>y</i> overflow	<i>x</i> overflow	<i>y</i> sign	<i>x</i> sign	1	Middle btn	Right btn	Left btn
byte 2	Mouse <i>x</i> movement							
byte 3	Mouse <i>y</i> movement							

Таблица №2. Пакет данных компьютерной мыши PS/2 состоящий из 3-х байт

Бит 3 первого байта равен 1, что указывает на то, что два следующих за ним байта являются частью 3-байтового пакета данных. Кроме того, первый байт содержит состояние трех кнопок в битах со 2 по 0, а также биты знака и переполнения для перемещения по осям *x* и *y*. Оставшиеся два байта содержат 8-битные части движений мыши по осям *x* и *y*.

Напишите программу на языке Си, которая инициализирует мышь и обрабатывает 3-байтовые пакеты перемещения, полученные от мыши. Ваша программа должна декодировать пакеты и отображать состояние кнопок на LEDR[2..0], изменение положения *x* на HEX[5..3] и изменение положения *y* на HEX[2..0].

Часть 4. Обработка прерываний от мыши

В этой части вы будете обрабатывать сообщения мыши, используя процедуру обработки прерываний. Процедура обработки прерываний (ISR) — это функция, которая выполняется при возникновении прерывания. Прерывание произойдет, когда бит RE в

управляющем регистре PS/2 будет установлен в значение 1 и на порт PS/2 поступит сообщение, готовое к считыванию.

Для этого измените свою программу на языке Си из части 3, реализовав четыре функции:

1. ***initialize_driver***
разрешить драйверу обрабатывать сообщения с помощью мыши. Эта функция должна перезагрузить мышь и разрешить прерывания для порта PS/2 на компьютере серии DE.
2. ***disable_driver***
отключить обработку сообщений с помощью мыши. Кроме того, отключите прерывания для порта PS/2.
3. ***get_mouse_change***
получите информацию о последних изменениях положения и кнопок мыши, о которых сообщает мышь.
4. ***PS2_ISR***
программа обслуживания прерываний, которая обрабатывает события, связанные с мышью.

Часть 5. Ограничение перемещений мыши в пределах экрана

В этой части вы улучшите драйвер вашего устройства, чтобы отслеживать местоположение мыши и ограничивать ее перемещения указанными границами. Для этого вы определите пять переменных: две для отслеживания текущего горизонтального и вертикального положения мыши, одна для отслеживания состояния кнопок мыши и две для указания максимальных горизонтальных и вертикальных координат, в которых может находиться мышь.

Вам необходимо создать драйвер устройства для мыши, содержащий следующие функции:

1. ***initialize_driver*** – как в части 4.
2. ***disable_driver*** – как в части 4.
3. ***set_mouse_bounds(max_x, max_y)***
установите максимальные координаты x и y, которые может иметь мышь.
4. ***get_mouse_state***
считывает текущее положение мыши на экране и состояние трех ее кнопок.
5. ***get_mouse_change*** – как в части 4.
6. ***PS2_ISR*** – как в части IV.

Напишите программу, которая отображает положение мыши, используя левые четыре hex-индикатора для отображения положения x и четыре правых для отображения положения y.

Установите максимальные координаты мыши x и y равными 319 и 239 соответственно.

Часть 6. Отображение указателя мыши на VGA

В этой части вы должны реализовать процедуры для отображения указателя мыши на экране. Для отображения указателя мыши на экране вы будете использовать VGA контроллер, который позволяет выводить изображение на экран компьютера с разрешением

320 на 240 пикселей. Изображение, отображаемое на экране, сохраняется в памяти и может быть легко изменено.

Чтобы нарисовать указатель мыши на экране, необходимо определить форму указателя, а также дополнительный буфер, содержащий содержимое экрана под указателем мыши. Вам следует использовать дополнительный буфер для сохранения части экрана, на которой отображается изображение мыши. Затем, когда мышь будет перемещена в будущем, вы сможете легко восстановить содержимое экрана и переместить мышь в новое место.

Чтобы определить форму указателя мыши, используйте матрицу из 16 строк и 8 столбцов. Каждая запись в матрице должна содержать одно из трех значений:

1. 0 – указывает, что соответствующий пиксель указателя мыши черный.
2. 1 – указывает, что соответствующий пиксель указателя мыши белый.
3. -1 – указывает, что соответствующий пиксель указателя мыши прозрачный.

Прозрачные пиксели не отображаются при наведении указателя мыши.

Матрица и изображение указателя мыши приведены на рисунке №2. Обратите внимание, что для обозначения прозрачных пикселей используется светло-голубой цвет.

0	-1	-1	-1	-1	-1	-1	-1
0	0	-1	-1	-1	-1	-1	-1
0	1	0	-1	-1	-1	-1	-1
0	1	1	0	-1	-1	-1	-1
0	1	1	1	0	-1	-1	-1
0	1	1	1	1	0	-1	-1
0	1	1	1	1	1	0	-1
0	1	1	1	1	0	0	0
0	1	1	1	0	-1	-1	-1
0	0	0	1	0	-1	-1	-1
0	-1	0	1	0	-1	-1	-1
-1	-1	-1	0	1	0	-1	-1
-1	-1	-1	0	1	0	-1	-1
-1	-1	-1	-1	0	1	0	-1
-1	-1	-1	-1	0	1	0	-1
-1	-1	-1	-1	-1	0	-1	-1

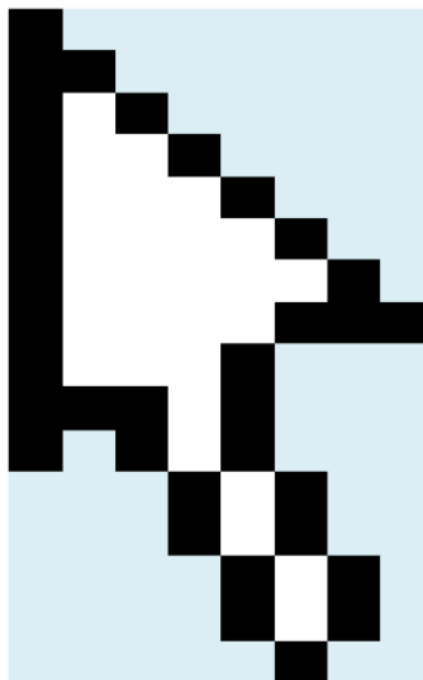


Рисунок №2. Матрица и графическое представление курсора компьютерной мыши.

Дополнительный буфер также должен состоять из 16 строк и 8 столбцов, в каждом из которых хранится цвет пикселя, на который будет наведен указатель мыши.

Адаптируйте ваш драйвер для перемещения указателя мыши по экрану. Определите две новые:

1. ***draw_mouse***

сохраняет изображение под указателем мыши и рисует его в текущем местоположении. Используйте матрицу, описанную выше, для рисования указателя на экране, предполагая, что верхний левый угол изображения указателя находится в текущем местоположении мыши. Эта функция должна быть видна только драйверу.

2. *erase_mouse*

удаляет мышь с экрана, перерисовав изображение под мышью. Эта функция должна быть видна только драйверу.

Измените процедуру обслуживания прерываний PS/2, чтобы при изменении положения мыши вызывались две вышеуказанные функции.

Запустите программу из части 5 и подключите VGA-выход платы DE-series к монитору компьютера. Вы должны иметь возможность видеть координаты указателя мыши на шестнадцатеричных дисплеях и сам указатель мыши на экране при перемещении мыши.

Часть 7*. Анимация и интерактивная игра с мышью

В этой части вы должны сочетать движение указателя мыши с анимацией на экране. Напишите программу для реализации простой игры, в которой заполненный прямоугольник перемещается и отскакивает от краев экрана. Игрок перемещает указатель мыши так, чтобы он находился на движущемся изображении прямоугольника, и в этот момент нажатие левой кнопки мыши должно изменить цвет прямоугольника и остановить его движение. Когда кнопка отпущена, прямоугольник должен снова начать двигаться. Игра станет интереснее, если вы будете увеличивать скорость или уменьшать размер прямоугольника после каждого удачного броска.

Чтобы выполнить анимацию, 30 раз в секунду вы должны прятать указатель мыши, стирать прямоугольник, рисовать прямоугольник в новом месте, а затем снова показывать указатель мыши. Вам необходимо определить две новые функции:

1. *hide_mouse*

Драйвер удалит мышь с экрана. Драйвер по-прежнему должен обрабатывать сообщения, поступающие от мыши, и обновлять ее положение и состояние кнопок, но указатель мыши не должен отображаться на экране.

2. *show_mouse*

позволяет драйверу отображать указатель мыши на экране. Если указатель ранее был скрыт, драйвер должен отображать указатель при вызове этой функции.

Чтобы отсчитать 1/30 секунды, мы можем воспользоваться тем, что контроллер VGA непрерывно перерисовывает изображение из памяти на экран, а один цикл перерисовки занимает 1/60 секунды. Мы можем попросить пиксельный буфер VGA сообщить нам, когда он завершит отрисовку последнего изображения. Ожидание двух таких циклов дает нам желаемую задержку в 30 секунд.

Когда вы запускаете свою программу, вы можете заметить, что при перемещении указателя мыши ее фрагменты остаются на экране несмотря на то, что вы тщательно стирали и перерисовывали их как в ISR, так и в основной программе. Причина в том, что во время скрытия/перерисовки мыши вашей основной программой может произойти прерывание. Для предотвращения этого вам потребуется настроить драйвер.

Контрольные вопросы

Теоретические вопросы

- Объясните назначение регистров данных и управления порта PS/2. Какие поля они содержат и как взаимодействуют с буфером FIFO?
- Какие этапы инициализации мыши через порт PS/2 необходимы для корректной работы? Опишите последовательность команд и ожидаемые ответы.

- Как обрабатываются 3-байтовые пакеты данных от мыши? Расшифруйте формат пакета, включая биты состояния кнопок, перемещения по осям и переполнения.
- В чём разница между опросом (polling) и обработкой прерываний (interrupt-driven) при работе с устройствами? Какие преимущества и недостатки у каждого подхода?
- Как организована процедура обработки прерываний (ISR) для порта PS/2? Какие функции она должна выполнять?
- Объясните принцип работы с VGA для отображения указателя мыши. Как реализованы функции draw_mouse и erase_mouse?
- Почему при анимации необходимо синхронизировать скрытие/отображение мыши с циклом обновления VGA? Какие проблемы могут возникнуть без такой синхронизации?

Практические вопросы

- Как декодировать 9-битные перемещения мыши из 8-битных полей в пакете данных?
- Каким образом ограничить перемещение мыши в пределах экрана (например, 320×240 пикселей)?
- Почему при повторном подключении мыши требуется повторная инициализация? Как ваша программа обрабатывает этот сценарий?
- Как реализовать прозрачные пиксели в матрице указателя мыши? Какие данные хранятся в дополнительном буфере?
- Какие меры предприняты для предотвращения артефактов на экране при одновременной работе ISR и основной программы?

Отчетные материалы

Исходный код

Программы на языке C для всех частей лабораторной работы.

Описание алгоритмов

Блок-схемы или псевдокод для ключевых функций:

- Инициализация мыши (часть 2).
- Обработка 3-байтовых пакетов (часть 3).
- Логика ISR для порта PS/2 (часть 4).
- Отрисовка/удаление указателя мыши (часть 6).

Демонстрация работы

Скриншоты/видео работы программы:

- Вывод данных на 7-сегментные индикаторы (часть 1).
- Отображение координат и состояния кнопок (часть 3).
- Указатель мыши на экране VGA (часть 6).
- Анимация с прямоугольником (часть 7*).

Выводы

- Роль драйверов устройств в абстракции аппаратного обеспечения.
- Сложности работы с низкоуровневыми интерфейсами (PS/2, VGA).
- Значение синхронизации и обработки прерываний в реальном времени.

Примечание

- Для части 7* (игра с анимацией) отчет может быть предоставлен по желанию, но его реализация добавит баллы.
- Все программы должны быть документированы (комментарии в коде, поясняющие ключевые части).